

# Supplemental Material: Beyond Task Completion: Auditing Evolving Tool Libraries in Agentic AI

Anonymous Author(s)

This supplement contains reproducibility material referenced from the main paper: the per-domain breakdown and the artifact gallery of representative synthesised tools. All numbers and artifacts are regenerated from `results_canonical/` by the scripts described in the main paper (`build_canonical.py`, `stats_report.py`, `make_tables.py`, `make_figures.py`).

## 1 Baseline Fairness Ledger

Several baselines are protocol *adaptations* rather than native ports. No-Evolution, One-Shot, and Code-Evol are native to this harness and omitted. All systems share the identical task stream, seed library, model (Sonnet 4), and verification function; only the tool-creation protocol varies.

| Baseline                 | Native setting                                                                         | Preserved faithfully                                                             | Adapted / fairness note                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Strategy-Only (EvoSkill) | Skill-library agent with NL strategy proposer/prompt generator                         | Skill Proposer and Prompt Generator prompt templates; strategy-follower strategy | Strategies injected as prompt hints, not ex-                                                                   |
| CREATOR-style            | Decision/execution split tool creation with reference tests                            | Two-phase create-then-use control flow, per-tool function synthesis              | ecutive; ChatGPT-style <i>strategy-level</i> evolution; not a leaderboard claim against EvoSkill.              |
| ToolCoder-style          | Code-generation-centric tool synthesis                                                 | Single-pass function generation from the task spec                               | Tool evolution, effectiveness varies substantially by problem type, a dimension invisible to a generated test. |
| ToolMaker-style          | Repo-grounded, Docker, OpenAI; diagnose/rewrite loop gated by provided reference tests | Two-stage diagnose-then-rewrite prompt; sandbox execution of auto-tests          | Runs on Bedrock/Claude rather than the native Model; same task stream and verifier for all systems.            |

Table 1: Baseline fairness ledger for adapted protocols.

## 2 ETS Weight Sensitivity

| Weights  | No-Evol | One-Shot | Str-Only | CREATOR | Code-Evol |
|----------|---------|----------|----------|---------|-----------|
| Original | .439    | .443     | .442     | .576    | .543      |
| Uniform  | .462    | .469     | .462     | .575    | .558      |
| TC-dom.  | .495    | .500     | .494     | .602    | .570      |
| LH-dom.  | .431    | .431     | .437     | .567    | .538      |
| LH=0     | .460    | .473     | .454     | .589    | .559      |

Table 2: ETS under alternative weight configurations (all Sonnet; multi-seed systems = mean of 4 seeds). CREATOR-style ranks first under all five schemes. **Weights** ( $w_{TC}$ ,  $w_{TQS}$ ,  $w_{RC}$ ,  $w_{LH}$ ): **Original** (.30, .20, .10, .40); **Uniform** (.25, .25, .25, .25); **TC-dom.** (.50, .20, .10, .20); **LH-dom.** (.20, .15, .10, .55); **LH=0** (.45, .35, .20, .00).

## 3 Skill-Bundle Evaluation

To extend the benchmark from bare-function evaluation to multi-file *skill bundles* (the format used by Anthropic Claude Skills and CoEvoSkills [?]), we wrap every preserved tool into a 4-file bundle (`SKILL.md`, `function`, `tests`, `metadata.json`) and score three bundle-level dimensions: Structure (*S*), Documentation (*D*), and Metadata (*M*). Bundle quality averages tool TQS with *S*, *D*, *M*.

| System                    | <i>n</i> | Bundle Q $\uparrow$ | <i>S</i> | <i>D</i> | <i>M</i> |
|---------------------------|----------|---------------------|----------|----------|----------|
| Code-Evol (sem. Q)        | 15       | <b>0.85</b>         | 1.00     | 0.84     | 1.00     |
| CREATOR-style             | 14       | 0.77                | 0.88     | 0.67     | 1.00     |
| One-Shot/Sonnet (3 seeds) | 9        | 0.69                | 0.88     | 0.50     | 1.00     |

Table 3: Skill-bundle quality (preserved tools only). The documentation dimension separates the systems most.

## 4 Per-Domain Analysis

| Domain      | Model  | TC   | Health | Ref. Ver. |
|-------------|--------|------|--------|-----------|
| Data Trans. | Sonnet | .573 | 14     | .734      |
| API Orch.   | Sonnet | .573 | 14     | .476      |
| API Orch.   | Haiku  | .621 | 3      | .333      |
| Numerical   | Sonnet | .651 | 4      | .667      |
| Numerical   | Haiku  | .621 | 3      | .556      |

Table 4: Code-Evol results by domain. API orchestration yields highest library health; data transformation creates the most tools but with lower quality.

*API orchestration: clean, composable problems.* Highest TC (up to 100%) and library health (77.8%), though based on a single session ( $N=11$  tasks). HMAC authentication and pagination are clean, single-responsibility functions well-suited to tool creation.

*Data transformation: high tool creation, lower quality.* Creates the most tools (14 on Sonnet) but achieves only 47.6% library health: many synthesised parsers for proprietary binary formats fail held-out tests while passing self-generated tests.

*Numerical computation: consistently difficult.* Both models achieve similar results (~63–65% TC, ~48–50% LH), suggesting these tasks challenge the LLM regardless of model capability.

## A Artifact Gallery

The benchmark preserves every synthesised tool to disk under `results_full/<config>/tools/<session>/<name>.py` (plus a `.meta.json` with the per-tool TQS scores). This appendix presents a side-by-side gallery of representative tools from the two tool-creating systems with multi-seed preserved source — Code-Evol (semantic- $Q$  run) and CREATOR-style — to make the per-tool failure modes the benchmark surfaces concrete. Per-dimension scores are shown inline ( $C$ ,  $R$ ,  $G$ ,  $Q$  and the composite TQS); bold indicates  $\geq 0.5$ .

### A.1 Code-Evol/Sonnet (semantic $Q$ )

15 tools were synthesised and preserved on disk across 9 sessions. Below we show 5 representative tools spanning the quality spectrum: correct exemplars, fragile high- $Q$ /low- $C$  tools, duplicates, and trivial (low-TQS) cases.

**Correct exemplar** (*passes hidden tests with  $C \geq 0.5$* )

**http\_get\_with\_headers** (*api\_orchestration\_s1*)  $C=0.50$   $R=0.50$   $G=0.50$   $Q=1.00$  TQS=0.62

```
def http_get_with_headers(url: str, headers: dict) -> str:
    """Make an HTTP GET request with custom headers."""
    import urllib.request
    import urllib.error

    try:
        # Create request object with custom headers
        request = urllib.request.Request(url)
        # ...
    except Exception as e:
        return f"Error: {str(e)}"
```

**Correct exemplar** (*passes hidden tests with  $C \geq 0.5$* )

**compute\_stats** (*numerical\_s1*)  $C=0.50$   $R=0.50$   $G=0.50$   $Q=1.00$  TQS=0.62

```
def compute_stats(data: list[float]) -> dict[str, float]:
    """Calculate basic statistical measures for a numerical dataset."""
    import math

    try:
        if not data:
            return {"error": "Empty dataset provided"}
        # ...
    except Exception as e:
        return {"error": f"Computation failed: {str(e)}"}
```

**Fragile** (*high  $Q$ , low  $C$  – looks well-formed, fails on hidden inputs*)

**hmac\_sha256** (*api\_orchestration\_s1*)  $C=0.00$   $R=0.50$   $G=0.00$   $Q=1.00$  TQS=0.38

```
def hmac_sha256(secret: str, message: str) -> str:
    """Generate HMAC-SHA256 hash for authentication and security purposes."""
    import hmac
    import hashlib

    try:
        # Handle None inputs explicitly
        if secret is None or message is None:
            # ...
    except Exception as e:
        return f"Error generating HMAC-SHA256: {str(e)}"
```

**Fragile** (*high  $Q$ , low  $C$  – looks well-formed, fails on hidden inputs*)

**compute\_crc32** (*data\_transform\_s5*)  $C=0.00$   $R=1.00$   $G=1.00$   $Q=1.00$  TQS=0.75

```
def compute_crc32(data: bytes) -> int:
    """Compute CRC32 checksum for data integrity verification."""
    import struct

    try:
        if not isinstance(data, bytes):
            return -1
        # ...
    except Exception:
        return -1
```

**Duplicate** (*synthesised across multiple sessions*)

**decode\_base64** (*data\_transform\_s5*)  $C=0.50$   $R=0.50$   $G=0.50$   $Q=1.00$  TQS=0.62

```
def decode_base64(encoded_string: str) -> bytes:
    """Decode base64 encoded binary data into bytes."""
    import base64

    # Validate input type
```

```
if not isinstance(encoded_string, str):
    raise TypeError(f"Expected string, got {type(encoded_string).__name__}")

# ...
# Return empty bytes on any decoding error (invalid base64, etc.)
return b''
```

## A.2 CREATOR-style/Sonnet

18 tools were synthesised and preserved on disk across 9 sessions. Below we show 5 representative tools spanning the quality spectrum: correct exemplars, fragile high- $Q$ /low- $C$  tools, duplicates, and trivial (low-TQS) cases.

**Correct exemplar** (*passes hidden tests with  $C \geq 0.5$* )

**decode\_guardian\_minimal\_data** (*data\_transform\_s5*)  $C=0.50$   $R=0.50$   $G=0.50$   $Q=0.89$   $TQS=0.60$

```
def decode_guardian_minimal_data(base64_data):
    """Decode minimal GUARDIAN data format handling single blocks and edge cases."""
    import base64
    import struct

    try:
        # Decode base64 data
        raw_data = base64.b64decode(base64_data)
        # ...
    except Exception as e:
        return {'text': '', 'integrity': False, 'blocks_processed': 0, 'details': f'Error: {str(e)}'}
```

**Correct exemplar** (*passes hidden tests with  $C \geq 0.5$* )

**decode\_qlog\_with\_continuations** (*data\_transform\_s3*)  $C=0.50$   $R=0.50$   $G=0.50$   $Q=0.89$   $TQS=0.60$

```
def decode_qlog_with_continuations(base64_data):
    """Decode QLOG data and merge continuation entries with their parent entries."""
    import base64
    import struct
    from datetime import datetime, timezone

    # Decode base64 data
    binary_data = base64.b64decode(base64_data)
    # ...

    return merged_entries
```

**Fragile** (*high  $Q$ , low  $C$  – looks well-formed, fails on hidden inputs*)

**fetch\_all\_users\_with\_pagination** (*api\_orchestration\_s1*)  $C=0.00$   $R=0.50$   $G=1.00$   $Q=0.89$   $TQS=0.60$

```
def fetch_all_users_with_pagination(base_url="http://127.0.0.1:18080/api/users"):
    """Utility: Fetches all users from a paginated API endpoint using cursor-based pagination.
    Continues fetching until has_more is False, collecting all user data across pages."""
    import urllib.request
    import urllib.parse
    import json

    all_users = []
    cursor = None
    # ...
    'names': names
}
```

**Fragile** (*high  $Q$ , low  $C$  – looks well-formed, fails on hidden inputs*)

**decode\_qlog\_binary\_data** (*data\_transform\_s3*)  $C=0.00$   $R=0.50$   $G=0.00$   $Q=0.78$   $TQS=0.32$

```
def decode_qlog_binary_data(base64_data):
    """Decode QLOG (Quantized Log Format) binary data into structured log records."""
    import base64
    import struct
    from datetime import datetime, timezone

    # Decode base64 data
    binary_data = base64.b64decode(base64_data)
    # ...

    return records
```

**Trivial** (*low overall TQS*)

**deserialize\_tpack** (*data\_transform\_s4*)  $C=0.00$   $R=0.50$   $G=0.00$   $Q=0.67$   $TQS=0.29$

```
def deserialize_tpack(base64_data):
    """Deserialize TPACK (Tagged Pack Format) binary data into Python objects."""
    import base64
    import struct
```

```
349     def parse_varint(data, offset):
350         result = 0
351         shift = 0
352         # ...
353         result, _ = parse_value(binary_data, 0)
354         return result
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
```

```
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
```